

MixedComplementarityProblems.jl: A Fast, Batched, Open-Source Interior-Point Solver for Mixed Complementarity Problems

David Fridovich-Keil

Department of Aerospace Engineering and Engineering Mechanics
The University of Texas at Austin
Austin, TX, USA
dfk@utexas.edu

Abstract—Mixed complementarity problems (MCPs) arise as the first-order optimality conditions of nonlinear programs and noncooperative games, and give a natural formulation for the multi-agent trajectory optimization problems that appear throughout robotics. The dominant solver for problems of this form is *PATH*, a mature but closed-source package. We present *MixedComplementarityProblems.jl*, an open-source, pure Julia implementation of an interior-point method for parametric MCPs that (i) matches *PATH*’s reliability on standard benchmarks and (ii) natively supports batched solving, in which many parameter instances are solved simultaneously and in parallel, either across CPU threads or on an NVIDIA GPU, behind a single solver implementation. On a multi-agent lane-change trajectory game representative of robotics planning problems, our CPU-multithreaded batched solver clears a batch of parametric instances 60–80 $\times$ faster than sequential calls to *PATH*, and our GPU backend clears the same batches roughly 20 $\times$ faster than *PATH*[**TODO: finalize the GPU-vs-CPU comparison for the trajectory game once the latest benchmark numbers are in—earlier runs showed GPU trailing CPU here, but that may no longer hold**]. We describe the solver’s interior-point formulation, the abstraction that lets a single solver implementation run unmodified across dense, batched-sparse, and single-large linear-algebra backends, and report benchmarks against *PATH* on both randomly generated quadratic programs and trajectory games.

I. INTRODUCTION

Multi-agent trajectory planning problems in robotics—lane changes and merges, intersection negotiation, racing, human-robot interaction—are naturally posed as noncooperative dynamic games, in which each agent optimizes its own trajectory subject to the others’ choices. The first-order (KKT) conditions of such a game, and of the constrained trajectory optimization problems that underlie it, take the form of a mixed complementarity problem (MCP). MCPs are therefore not a peripheral formalism for this class of problems; they are the computational object that planners in this space ultimately have to solve, often many times per planning cycle.

The dominant solver for problems of this form is *PATH* [1], accessed from Julia through the *PATHSolver.jl/ParametricMCPs.jl* stack. *PATH* is mature and reliable, but it is closed-source, solves one parameter instance at a time, and was not designed to sit inside a differentiable pipeline or underneath an outer loop that itself needs to be parallelized. That is a real limitation for the workloads robotics increasingly relies on: scenario- or sampling-based planning, receding-horizon replanning across many candidate initial conditions, and learning formulations

that differentiate through a game’s equilibrium. Each of these needs many structurally identical MCPs solved together, not one instance solved in isolation.

No existing tool combines an open, modifiable solver with native support for solving many such instances at once, in parallel, on the hardware a robotics practitioner already has. Interior-point methods are a natural fit for closing this gap: they are simple enough to reimplement and adapt to a new application, they are straightforward to differentiate through, and their dominant per-iteration cost—a linear solve against the KKT system—parallelizes cleanly across a batch of instances that share a common sparsity structure, whether that parallelism is spread across CPU threads or onto a GPU.

We present *MixedComplementarityProblems.jl*, an open-source, pure Julia interior-point solver for parametric MCPs. Concretely, we contribute:

- An interior-point method for parametric MCPs that matches *PATH*’s reliability on standard benchmarks, while remaining a compact, modifiable, pure Julia implementation that differentiates through its own solutions via *ChainRulesCore/ForwardDiff*.
- A native *batched* solve mode, in which many parameter instances sharing one MCP structure are solved simultaneously, parallelized across CPU threads or an NVIDIA GPU behind a single solver implementation.
- An abstraction boundary—a small verb set (residual, Jacobian assembly, factorize, solve) between the interior-point loop and its linear-algebra backend—that lets that one implementation run unmodified across dense, batched-sparse, and single-large-instance regimes, so that supporting a new device or linear-algebra strategy never requires touching the solver loop itself.

As Fig. 1 previews and Section V quantifies, on a multi-agent lane-change trajectory game representative of robotics planning problems, clearing a batch of parametric instances with our batched solver takes a small fraction of the wall-clock time required to clear the same batch with sequential calls to *PATH*. The rest of this report is organized as follows. Section II reviews MCPs and their connection to KKT conditions and games. Section III describes the interior-point method and its strategy/device abstraction. Section IV details the batched CPU and GPU backends. Section V reports benchmarks against *PATH* on quadratic programs and trajectory games.

[TODO: replace with rendered lane-change trajectory game schematic, showing the two vehicles' solved trajectories overlaid with a faint fan of trajectories from other initial conditions in the same batch]

(a) One multi-agent trajectory game is one instance out of a batch of parametrically related MCPs, solved simultaneously.

[TODO: replace with wall-clock-vs-batch-size plot: PATH (serial), CPU-batched, and GPU-batched, log-scaled y -axis]

(b) Clearing that batch: sequential PATH calls versus our batched CPU and GPU solves.

Fig. 1. **[TODO: finalize caption once the real figure is in]** Solving a batch of parametric trajectory-game instances (a) with MixedComplementarityProblems.jl's batched interior-point solver is substantially faster than solving the same batch with sequential calls to PATH (b).

Section VI discusses limitations and future work.

II. BACKGROUND: MIXED COMPLEMENTARITY PROBLEMS

TODO

III. SOLVER DESIGN

TODO

IV. BATCHED AND GPU-PARALLEL SOLVING

TODO

V. EXPERIMENTS

TODO

VI. DISCUSSION AND FUTURE WORK

TODO

REFERENCES

- [1] S. P. Dirkse and M. C. Ferris, "The path solver: a nonmonotone stabilization scheme for mixed complementarity problems," *Optimization methods and software*, vol. 5, no. 2, pp. 123–156, 1995.